

Τμήμα Μηχανικών Η/Υ & Πληροφορικής
ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ
Πανεπιστήμιο Ιωαννίνων

ΜΥΕ023–ΠΑΡΑΛΛΗΛΑ ΣΥΣΤΗΜΑΤΑ ΚΑΙ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ

1ο ΣΕΤ ΑΣΚΗΣΕΩΝ (OpenMP)

ΔΗΜΗΤΡΑ ΧΡΙΣΤΙΝΑ ΓΚΑΡΑΒΕΛΑ ΑΜ:5051
cs05051@uoi.gr

Η πρώτη εργασία αφορά τον παράλληλο προγραμματισμό με το μοντέλο κοινόχρηστου χώρου διευθύνσεων μέσω του προτύπου OpenMP. Η αλλαγή του αριθμού των νημάτων γινόταν με την εντολή `export OMP_NUM_THREADS` στο τερματικό.

Όλες οι μετρήσεις έγιναν στο παρακάτω σύστημα:

Όνομα υπολογιστή	opti7020ws24
Επεξεργαστής	Intel i5-4590
Πλήθος πυρήνων	4
Μεταφραστής	gcc v11.2.0 (Ubuntu)

Άσκηση 1

Το πρόβλημα

Στην παρούσα άσκηση υλοποίησα μια εφαρμογή για τον υπολογισμό των πρώτων αριθμών μέχρι το 10.000.000. Συγκεκριμένα, παραλληλοποίησα ένα υπάρχον σειριακό πρόγραμμα χωρίς να τροποποιήσω τον βασικό αλγόριθμο, απλώς διαμοιράζοντας σωστά τη δουλειά μεταξύ των νημάτων. Επιπλέον, πραγματοποίησα πειράματα με διαφορετικές μεθόδους διαμοιρασμού (schedules) και διαφορετικές ρυθμίσεις reduction στις κρίσιμες global μεταβλητές.

Μέθοδος παραλληλοποίησης

Χρησιμοποιήθηκε το σειριακό πρόγραμμα από την ιστοσελίδα του μαθήματος. Για την υλοποίηση της συνάρτησης `openmp_primes()` προστέθηκε η οδηγία **`#pragma omp parallel for private(num, divisor, quotient, remainder)`** πριν από τον βρόχο `for` που επεξεργάζεται κάθε περιττό αριθμό. Με την χρήση της **`#pragma omp parallel for`** ενημερώνουμε τον μεταγλωττιστή ότι ο επόμενο βρόχος `for` θα πρέπει να εκτελείται παράλληλα, με το πλήθος των νημάτων που έχει οριστεί (**`OMP_NUM_THREADS`**). Η προσθήκη του **`private(num, divisor, quotient, remainder)`** εξασφαλίζει ότι κάθε νήμα που συμμετέχει στην εκτέλεση του βρόχου θα διαθέτει το δικό του αντίγραφο των μεταβλητών `num`, `divisor`, `quotient` και `remainder`. Οι μεταβλητές αυτές υπολογίζονται ξεχωριστά σε κάθε επανάληψη του βρόχου, χωρίς το `private` μπορεί να προκύψει το πρόβλημα των race conditions, όπου δύο ή περισσότερα νήματα θα προσπαθούν ταυτόχρονα να τροποποιήσουν την ίδια μεταβλητή, οδηγώντας σε εσφαλμένα αποτελέσματα. Η αρχική αυτή υλοποίηση απέδιδε εσφαλμένα αποτελέσματα στην αθροιστική μεταβλητή `count`, οπότε προστέθηκε το **`reduction(+: count)`**. Έτσι, κάθε νήμα να συσσωρεύει τοπικά τις αυξήσεις για την μεταβλητή `count` και, στο τέλος, να γίνεται η σωστή συνολική αθροιστική συσσώρευση. Στη συνέχεια, πρόσθεσα scheduling για να εξισορροπήσει το φορτίο εργασίας ανάμεσα στα νήματα έτσι ώστε να

βελτιωθεί η συνολική απόδοση και δοκίμασα διάφορες παραλλαγές του. Στις δοκιμές με **schedule(guided,5)** και **schedule(static,5)** παρατηρήθηκε ότι το lastprime δεν υπολογιζόταν σωστά, και ως εκ τούτου προστέθηκε το **reduction(max: lastprime)**. Έτσι, εξασφαλίζεται ότι το τελικό lastprime θα είναι το μεγαλύτερο, δηλαδή το τελευταίο πρώτο που θα έπρεπε να έχει υπολογιστεί. Συμπερασματικά, ο σωστός συνδυασμός που απέδιδε ορθά αποτελέσματα (σωστό πλήθος πρώτων και σωστό τελευταίο πρώτο) ήταν ο συνδυασμός των 2 reductions (σε count και lastprime) σε συνδυασμό με το schedule()).

Πειραματικά αποτελέσματα – μετρήσεις

Τα προγράμματα εκτελέστηκαν στο σύστημα που αναφέραμε στην εισαγωγή και η χρονομέτρηση έγινε με τη συνάρτηση **omp_get_wtime()**. Συγκεκριμένα, αποθηκεύεται η τρέχουσα τιμή του χρόνου στην μεταβλητή start μέσω της κλήσης **omp_get_wtime()**, εκτελείται η συνάρτηση **serial_primes(UPTO)**, η οποία υπολογίζει τους πρώτους αριθμούς μέχρι το 10.000.000, εκτελείται πάλι η **omp_get_wtime()** για να αποθηκευτεί ο χρόνος στο τέλος της λειτουργίας στην μεταβλητή end και η διαφορά end - start αποθηκεύεται στην μεταβλητή serial_time, η οποία αντιπροσωπεύει το χρόνο που χρειάστηκε η εκτέλεση της **serial_primes(UPTO)** σε δευτερόλεπτα. Η αντίστοιχη διαδικασία εκτελείται για την μέτρηση του χρόνου της παράλληλης υλοποίησης.

Ο χρόνος εκτέλεσης της αρχικής σειριακής υλοποίησης αποτέλεσε τη γραμμή βάσης για την αξιολόγηση της βελτίωσης που επιφέρουν οι παράλληλες εκδόσεις. Η μέτρηση έγινε με τη χρήση της **omp_get_wtime()** και το αποτέλεσμα ήταν περίπου 13.0 δευτερόλεπτα, τιμή που είναι παρόμοια με την παράλληλη εκτέλεση όταν χρησιμοποιείται ένα μόνο νήμα.

Χρησιμοποιήσαμε από 1 μέχρι 4 νήματα, και εκτελέσαμε 4 διαφορετικά πειράματα. Κάθε πείραμα διαφέρει στο scheduling, δηλαδή static, static,5 , dynamic,5 , guided,5 για τα πειράματα 1,2,3 και 4 αντίστοιχα.

Κάθε πείραμα εκτελέστηκε 4 φορές και υπολογίστηκαν οι μέσοι όροι.

#pragma omp parallel for private(num, divisor, quotient, remainder) reduction(+: count) reduction(max:lastprime):

- **schedule(static)**

Νήματα	1 ^η Εκτέλεση	2 ^η Εκτέλεση	3 ^η Εκτέλεση	4 ^η Εκτέλεση	5 ^η Εκτέλεση	Μέσος Όρος
1	13.888070	13.907341	13.908698	13.914195	13.107988	13.745258
2	8.320111	8.523202	8.724304	8.729387	8.317114	8.522824
3	5.787381	5.788759	5.794120	5.788233	5.916377	5.814974

4	5.538872	7.147762	7.449908	6.884934	4.504010	6.305097
---	----------	----------	----------	----------	----------	----------

- **schedule(static,5)**

Νήματα	1 ^η Εκτέλεση	2 ^η Εκτέλεση	3 ^η Εκτέλεση	4 ^η Εκτέλεση	5 ^η Εκτέλεση	Μέσος Όρος
1	13.926298	13.927806	13.926338	13.148621	13.150911	13.615995
2	6.742468	6.742130	7.016137	7.014898	6.742186	6.851564
3	5.136572	5.133274	5.136422	5.139586	5.129956	5.135162
4	4.732564	4.718738	4.792205	4.872850	4.670639	4.757399

- **schedule(dynamic)**

Νήματα	1 ^η Εκτέλεση	2 ^η Εκτέλεση	3 ^η Εκτέλεση	4 ^η Εκτέλεση	5 ^η Εκτέλεση	Μέσος Όρος
1	13.170395	13.873306	13.851878	13.842183	13.841613	13.715875
2	6.764481	6.764217	6.762972	6.765343	6.763535	6.764110
3	4.620149	4.621014	4.624822	4.623064	4.620508	4.621911
4	3.463335	3.459762	4.623792	4.623684	4.623687	4.158852

- **schedule(dynamic,5)**

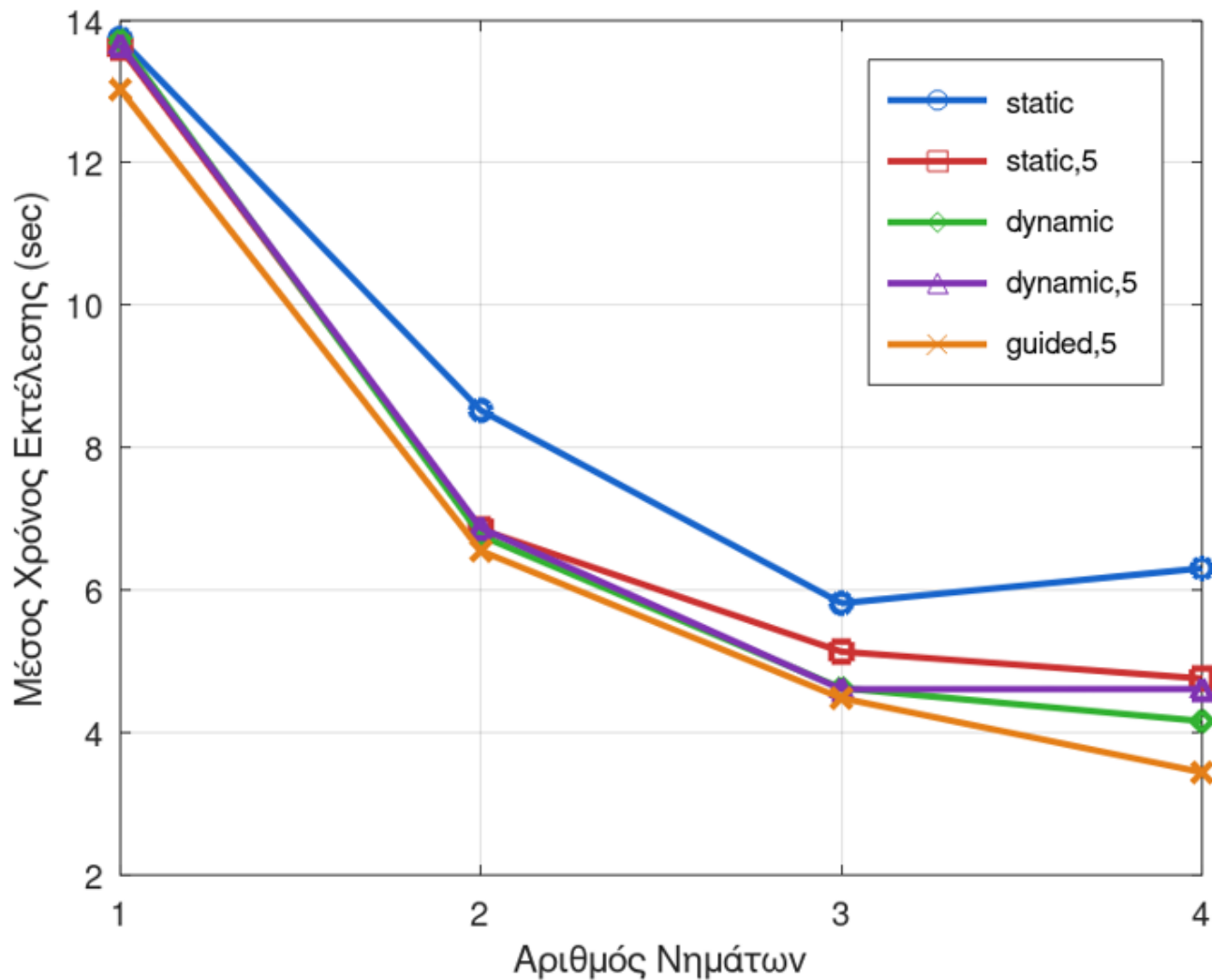
Νήματα	1 ^η Εκτέλεση	2 ^η Εκτέλεση	3 ^η Εκτέλεση	4 ^η Εκτέλεση	5 ^η Εκτέλεση	Μέσος Όρος
1	13.817943	13.821707	13.877135	13.705693	13.044208	13.653337
2	6.743806	6.744751	6.744858	7.049699	7.074493	6.871521
3	4.604766	4.605058	4.610001	4.610893	4.605322	4.607208
4	4.612413	4.609731	4.607724	4.608429	4.608810	4.609421

- **schedule(guided,5)**

Νήματα	1 ^η Εκτέλεση	2 ^η Εκτέλεση	3 ^η Εκτέλεση	4 ^η Εκτέλεση	5 ^η Εκτέλεση	Μέσος Όρος
1	13.032689	13.030575	13.035801	13.020634	13.034354	13.030811
2	6.549994	6.550489	6.549507	6.546995	6.557167	6.550830
3	4.482660	4.481665	4.482437	4.483208	4.482343	4.482463
4	3.439631	3.439616	3.443030	3.439691	3.442582	3.440910

Με βάση τους πίνακες των αποτελεσμάτων προκύπτει η παρακάτω γραφική παράσταση.

Σύγκριση Schedules ως προς Χρόνο Εκτέλεσης



Σχόλια

Από τη γραφική παράσταση παρατηρούμε ότι όσο αυξάνεται ο αριθμός των νημάτων (από 1 έως 4), ο μέσος χρόνος εκτέλεσης μειώνεται σε όλες τις περιπτώσεις scheduling.

Ειδικότερα:

- Το `schedule(static)` εμφανίζει τη χειρότερη απόδοση με 4 νήματα, καθώς ο χρόνος αυξάνεται ελαφρώς αντί να συνεχίσει να μειώνεται.
- Οι επιλογές `schedule(static,5)` και `schedule(dynamic,5)` παρουσιάζουν πολύ πιο ομαλή μείωση, αλλά όχι την καλύτερη επίδοση.

- Η καλύτερη επίδοση επιτυγχάνεται με το `schedule(guided,5)`, ειδικά με 4 νήματα, όπου φτάνει περίπου τα 3.44 δευτερόλεπτα.
- Το `schedule(dynamic)` έχει καλή γενικά συμπεριφορά, αλλά για 4 νήματα παρατηρείται απότομη αύξηση στον χρόνο, που πιθανότατα οφείλεται σε ανισοκατανομή των επαναλήψεων.

Η διαφορά στις επιδόσεις οφείλεται κυρίως στον τρόπο με τον οποίο κατανέμονται οι επαναλήψεις στους διαθέσιμους πυρήνες. Πιο συγκεκριμένα:

- Το `static scheduling` κατανέμει ισάριθμες επαναλήψεις προκαταβολικά, χωρίς να λαμβάνει υπόψη το μεταβλητό κόστος κάθε επανάληψης. Αυτό οδηγεί σε ανισοκατανομή φόρτου, ειδικά σε επαναλήψεις που χρειάζονται περισσότερους υπολογισμούς (όπως οι μεγαλύτεροι αριθμοί).
- Αντίθετα, οι επιλογές `dynamic` και `guided scheduling` επιτρέπουν πιο ευέλικτη κατανομή, αναθέτοντας επιπλέον επαναλήψεις στα νήματα μόλις αυτά ολοκληρώσουν τις προηγούμενες. Έτσι, επιτυγχάνεται καλύτερη εκμετάλλευση των πόρων και εξισορρόπηση του φόρτου.
- Το `guided scheduling` έχει το πρόσθετο πλεονέκτημα ότι αρχικά αναθέτει μεγαλύτερα κομμάτια εργασίας και στη συνέχεια μικρότερα, μειώνοντας έτσι το `overhead` και βελτιστοποιώντας τη χρήση των νημάτων.

Συμπερασματικά, το `schedule(guided,5)` προσφέρει τη βέλτιστη ισορροπία ανάμεσα σε ευελιξία και αποδοτικότητα, δίνοντας τον μικρότερο χρόνο εκτέλεσης για 4 νήματα, ενώ το `static` είναι η λιγότερο αποτελεσματική επιλογή λόγω κακής εξισορρόπησης του φόρτου.

Η άσκηση εκτελέστηκε με τις εντολές:

```
gcc -fopenmp primes.c -o primes
```

```
export OMP_NUM_THREADS=x , όπου x=1,2,3,4.
```

```
./primes
```

Άσκηση 2

Το πρόβλημα

Στην άσκηση αυτή υλοποίησα μια εφαρμογή Gaussian Blur για εικόνες BMP, με σκοπό την θόλωση της εικόνας βάσει μίας καθορισμένης ακτίνας θόλωσης (radius). Αρχικά, χρησιμοποιήθηκε η σειριακή υλοποίηση της συνάρτησης `gaussian_blur_serial()` που μας δίνεται. Στη συνέχεια, παραλληλοποίησα την ίδια ακριβώς λογική χρησιμοποιώντας OpenMP με δύο διαφορετικές προσεγγίσεις: α) με παραλληλοποίηση των βρόχων μέσω `#pragma omp parallel for` και β) με δημιουργία `tasks`, όπου κάθε γραμμή της εικόνας αποτελεί ένα ξεχωριστό `task`. Όλες οι εκτελέσεις έγιναν στο ίδιο σύστημα με 4 πυρήνες. Αρχικά μετρήθηκε ο χρόνος εκτέλεσης της σειριακής υλοποίησης `gaussian_blur_serial()`, ο οποίος αποτέλεσε τη βάση για τη σύγκριση με τις παράλληλες εκδόσεις. Ο μέσος χρόνος εκτέλεσης με 1 νήμα ήταν 18.28 δευτερόλεπτα, και όλες οι μετέπειτα παρατηρήσεις έγιναν σε σχέση με αυτόν.

Μέθοδος παραλληλοποίησης

Για την παραλληλοποίηση μέσω **loops**, τοποθετήσαμε την οδηγία: **`#pragma omp parallel for`** πάνω από τον εξωτερικό βρόχο που σαρώνει τις γραμμές της εικόνας (μεταβλητή `i`). Έτσι, κάθε νήμα μπορεί να επεξεργάζεται διαφορετικές γραμμές (rows) της εικόνας ταυτόχρονα. Η χρήση της **`private()`** για τις μεταβλητές που επαναχρησιμοποιούνται σε κάθε επανάληψη των βρόχων ή ανά `task` είναι απαραίτητη για την αποφυγή παρεμβολών μεταξύ των νημάτων. Όταν μια μεταβλητή δηλώνεται ως `private`, κάθε νήμα ή `task` δημιουργεί το δικό του ξεχωριστό αντίγραφο αυτής της μεταβλητής, το οποίο χρησιμοποιείται αποκλειστικά στο πλαίσιο της εκτέλεσής του.

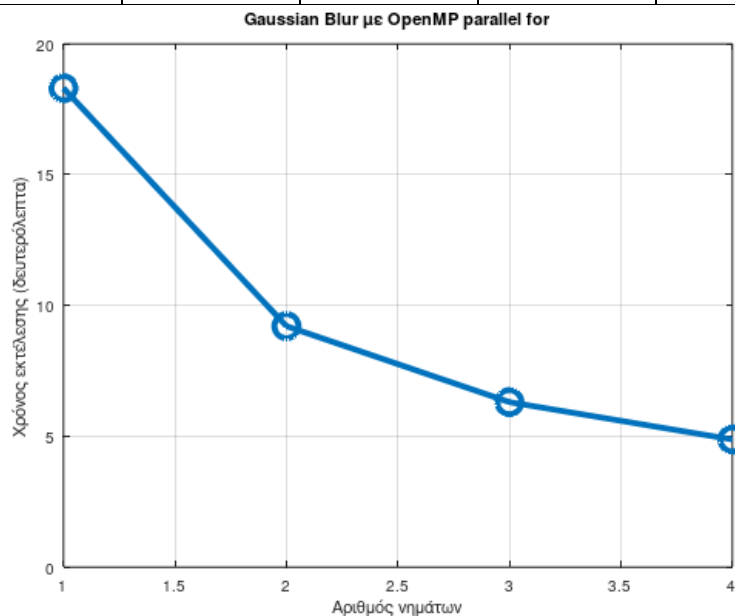
Στην παραλληλοποίηση με OpenMP **tasks**, χρησιμοποιήθηκε το μοντέλο των OpenMP `tasks`. Η λογική ήταν να δημιουργείται ένα `task` για κάθε γραμμή της εικόνας. Η χρήση της οδηγίας **`#pragma omp single nowait`** διασφαλίζει ότι μόνο ένα νήμα από το σύνολο των διαθέσιμων θα εκτελέσει το μπλοκ κώδικα στο οποίο δημιουργούνται τα `tasks` (δηλαδή ο βρόχος `for` που παράγει ένα `task` για κάθε γραμμή της εικόνας). Αυτό αποτρέπει την πολλαπλή δημιουργία των ίδιων `tasks` από διαφορετικά νήματα, κάτι που θα μπορούσε να οδηγήσει σε λάθη ή διπλή επεξεργασία. Χωρίς το **`nowait`** τα υπόλοιπα νήματα θα περίμεναν παθητικά μέχρι να ολοκληρωθεί πλήρως η δημιουργία όλων των `tasks`. Αντίθετα, με το **`nowait`**, τα υπόλοιπα νήματα μπορούν να ξεκινήσουν αμέσως την εκτέλεση των ήδη δημιουργημένων `tasks`, χωρίς να περιμένουν την ολοκλήρωση της δημιουργίας ολόκληρης της λίστας. Η εντολή **`firstprivate(i)`** διασφαλίζει ότι κάθε `task` "κλειδώνει" σωστά τη γραμμή

της εικόνας που πρέπει να επεξεργαστεί, διατηρώντας την τιμή του i από το σημείο δημιουργίας του task. Επιπλέον, ο ορισμός των μεταβλητών υπολογισμού (j , row, col, redSum, greenSum, blueSum, weightSum) ως **firstprivate** είναι κρίσιμος για τη σωστή λειτουργία και την επίτευξη πλήρους παραλληλίας, καθώς αποφεύγονται συγκρούσεις και ανάγκες συγχρονισμού μεταξύ των tasks.

Πειραματικά αποτελέσματα – μετρήσεις

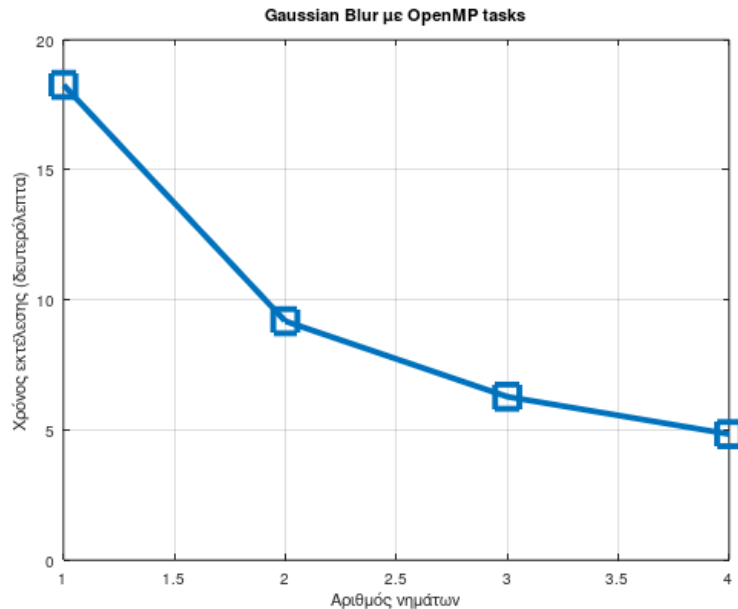
- `gaussian_blur_omp_loops()`

Νήματα	1 ^η Εκτέλεση	2 ^η Εκτέλεση	3 ^η Εκτέλεση	4 ^η Εκτέλεση	5 ^η Εκτέλεση	Μέσος Όρος
1	18.298925	18.310319	18.310359	18.303054	18.311965	18.306924
2	9.215120	9.206212	9.220222	9.209441	9.211628	9.212125
3	6.308752	6.305292	6.313381	6.305628	6.308990	6.308009
4	4.877455	4.864285	4.879797	4.875447	4.902212	4.879839



- `gaussian_blur_omp_tasks()`

Νήματα	1 ^η Εκτέλεση	2 ^η Εκτέλεση	3 ^η Εκτέλεση	4 ^η Εκτέλεση	5 ^η Εκτέλεση	Μέσος Όρος
1	18.266747	18.269152	18.273466	18.263462	18.263397	18.267645
2	9.184352	9.183829	9.179924	9.183321	9.184177	9.183521
3	6.283108	6.281057	6.282099	6.281450	6.282321	6.282007
4	4.834330	4.889214	4.831257	4.837044	4.826300	4.843229



Σχόλια

Παρατηρούμε ότι, και οι δύο μέθοδοι παρουσιάζουν σημαντική βελτίωση στην απόδοση καθώς αυξάνεται ο αριθμός των νημάτων από 1 σε 4 και φυσικά από την σειριακή υλοποίηση. Το γράφημα για την υλοποίηση με OpenMP parallel for δείχνει ομαλή μείωση του χρόνου εκτέλεσης με κάθε αύξηση των νημάτων. Η υλοποίηση με OpenMP tasks παρουσιάζει παρόμοια βελτίωση, όμως σε όλες τις περιπτώσεις αποδίδει ελαφρώς καλύτερα από την αντίστοιχη με parallel for, ειδικά στα 3 και 4 νήματα.

Με βάση τα αποτελέσματα παρατηρείται ότι η αύξηση του αριθμού των νημάτων μειώνει αισθητά τον χρόνο εκτέλεσης, ιδιαίτερα μέχρι τα 3 νήματα. Αν και η μείωση δεν είναι απολύτως γραμμική (δηλαδή δεν ισχύει ακριβώς ότι 2 νήματα εκτελούν 2 φορές γρηγορότερα), οι αποδόσεις δείχνουν ικανοποιητικό βαθμό παραλληλίας, ειδικά με χρήση OpenMP tasks όπου η δυναμική κατανομή εργασιών βοηθά στην καλύτερη αξιοποίηση των διαθέσιμων πόρων. Η εκτέλεση του αρχείου έγινε με τις παρακάτω εντολές:

```
gcc -fopenmp -o gaussian-blur gaussian-blur.c -lm
```

```
export OMP_NUM_THREADS= x , όπου x=1,2,3,4
```

```
./gaussian-blur 8 1500.bmp
```

Η θολωμένη εικόνα είναι η παρακάτω:

